“Bog‘lash” degani nima?

Vue’da bog‘lash deganda bir qiymatni uni ko‘rsatadigan joyga yoki bir hodisani unga javob beradigan funksiyaga ulash tushuniladi. Siz aloqani kodda yozasiz; Vue esa tegishli o‘zgarishlarni interfeysga yetkazadi.

Avval uchta narsani ajratamiz

Holat (state) - dastur eslab turadigan qiymat. Masalan: sarlavha, yangi experience ID si yoki forma ochiqmi degan boolean.

Ko‘rinish (UI) - foydalanuvchi ko‘radigan sarlavha, tugma va forma.

Hodisa (event) - nimadir sodir bo‘lgani haqidagi xabar. Masalan: tugma bosildi yoki formani yopish so‘raldi.

```
const isOpen = ref(false)

// Template ichida:
<UCollapsible :open="isOpen" />
```

Bu misolda isOpen - holat, UCollapsible - uni ko‘rsatadigan komponent, :open esa ular orasidagi aloqa. isOpen true bo‘lsa, komponentga open=true uzatiladi.

> Bog‘lash ma’lumotlar bazasiga saqlash degani emas. U komponentlar va interfeys orasida qiymat yoki hodisa qanday uzatilishini belgilaydi.

Sizning komponentlaringiz

```
ExperienceSection.vue          Parent (ota komponent)
        |
        +-- ResumeEditorSection.vue    Child (bola)
                    |
                    +-- UCollapsible   Ichki child
```

Parent va child nisbiy tushunchalar: ResumeEditorSection o‘z parentiga nisbatan child, UCollapsible uchun esa parent hisoblanadi.

Keyingi sahifalarda avval oddiy misollarni ko‘ramiz, so‘ng yangi experience qo‘shilganda uning formasi qanday ochilishini bosqichma-bosqich tahlil qilamiz.

Misollar Vue 3 va <script setup lang="ts"> uchun. Oddiy Vue loyihasida ref, watch va toRef vue paketidan import qilinadi; Nuxt ularni odatda avtomatik import qiladi.

Reaktivlik va template yozuvlari

ref: Vue kuzata oladigan qiymat

```
import { ref } from 'vue'

const title = ref('Ish tajribasi')

// Script ichida qiymatni o‘zgartirish:
title.value = 'Yangi tajriba'
```

ref qiymatni reaktiv qobiq ichida saqlaydi. Shu qiymat template ichida ishlatilgan bo‘lsa, o‘zgarganida Vue kerakli ko‘rinishni yangilaydi. Oddiy let o‘zgaruvchini almashtirishning o‘zi bunday yangilanishni ta’minlamaydi.

```
<h2>{{ title }}</h2>
<MySection :section_header="title" />
```

Bu yerda {{ title }} qiymatni matn sifatida chiqaradi. :section_header="title" esa qiymatni komponentning propiga beradi. Template ichida yuqoridagi ref avtomatik ochiladi, shuning uchun title.value yozish shart emas.

Bir xil ko‘rinadigan, ammo turlicha ishlaydigan yozuvlar

```
<MySection section_header="title" />
<MySection :section_header="title" />
```

Birinchi qatorda childga aynan “title” matni keladi. Ikkinchi qatorda title o‘zgaruvchisining qiymati, masalan “Yangi tajriba”, keladi. Ikki nuqta - v-bind ning qisqa yozilishi.

```
<button :disabled="isCreating" @click="save">
    Saqlash
</button>
```

:disabled - tugma xususiyatini qiymatga bog‘laydi. @click - bosilish hodisasini save funksiyasiga bog‘laydi. @ belgisi v-on ning qisqa yozilishi.

> “:” bilan qiymat berasiz. “@” bilan hodisani tinglaysiz. “v-model” bilan qiymat va uni yangilash hodisasini birga ulaysiz.

Yangilanish qachon ko‘rinadi?

Reaktiv qiymat scriptda yangilanadi, DOM yangilanishlari esa Vue tomonidan navbatga yig‘iladi. O‘zgargan elementga focus berish yoki uning o‘lchamini olish kerak bo‘lsa, nextTick kerak bo‘lishi mumkin. Faqat open holatini almashtirish uchun uni qo‘shish shart emas.

Props: qiymat parentdan childga

Prop - child tashqaridan qabul qiladigan parametr. defineProps child qaysi parametrlarni kutishini e’lon qiladi. Haqiqiy qiymatni parent template orqali beradi.

Parent: qiymatni saqlaydi va uzatadi

```
const heading = ref('Ish tajribasi')

// Template:
<ResumeEditorSection :section_header="heading" />
```

Child: qiymatni qabul qiladi va ko‘rsatadi

```
const props = defineProps<{
    section_header: string
}>()

// Template:
<h2>{{ props.section_header }}</h2>

heading  -->  section_header  -->  ekrandagi sarlavha
```

heading va section_header nomlari bir xil bo‘lishi shart emas. Chapdagi section_header - childdagi prop nomi; o‘ngdagi heading - parentdagi ifoda.

Parent heading ni o‘zgartirsa, child yangi prop qiymatini oladi. Child props.section_header ga qiymat tayinlamaydi: yangilash kerak bo‘lsa, parentga hodisa yuboradi.

Sizdagi last_item_id

```
// Parent:
const last_item_id = ref<string | null>(null)

// Parent template:
<ResumeEditorSection :last_item_id="last_item_id" />

// Childdagi props turi ichida:
last_item_id?: string | null
```

Bu aloqada yangi ID parentdan childga keladi. Ammo propning o‘zi openedForms ga hech narsa qo‘shmaydi. ID kelganda nima qilish kerakligini watch bilan alohida belgilaysiz.

> “?” prop uzatilmasligi ham mumkinligini bildiradi. Boshqa sectionlar last_item_id bermasa, optional prop mos keladi. Callback null va undefined holatlarini tekshirishi kerak.

Object va array props ichki ma’lumotlari umumiy reference orqali o‘zgarishi mumkin. Tushunarli ma’lumot oqimi uchun ularni yangilash mas’uliyatini ham aniq saqlang.

Manba: Vue: Props

Emit: child parentga xabar beradi

emit - komponent hodisasini chiqarish. Parent shu hodisani tinglab, o‘z funksiyasini ishga tushiradi. defineEmits esa qanday hodisalar va argumentlar mavjudligini e’lon qiladi.

Childdagi tugma

```
const emit = defineEmits<{
    add: []
    delete: [id: string]
}>()

// Template:
<button @click="emit('add')">Qo‘shish</button>
```

Parentdagi tinglovchi

```
<ResumeEditorSection @add="handleAddSection" />

Tugma bosildi -> emit("add") -> handleAddSection()
```

add: [] hodisa qo‘shimcha argument yubormasligini bildiradi. delete: [id: string] esa o‘chirish hodisasi string ID yuborishini bildiradi.

```
// Child:
emit("delete", "exp-42")

// Parent template:
<ResumeEditorSection @delete="deleteExperience" />

// Natija: deleteExperience("exp-42") chaqiriladi.
```

$event nimani anglatadi?

```
<ResumeEditorSection
    @delete="deleteExperience($event)"
/>
```

Bu misolda $event - child yuborgan "exp-42" qiymati. Oddiy HTML click hodisasida esa u brauzerning hodisa obyekti bo‘ladi. Qiymat turi qaysi hodisa tinglanayotganiga bog‘liq.

emit ma’lumotni saqlamaydi va server so‘rovini o‘zi bajarmaydi. Parentning hodisa handleri bajaradigan ishni siz belgilaysiz.

Asinxron create haqida

Parentdagi handleAddSection async bo‘lishi mumkin. Child emit chaqiruvini await qilish orqali parent yaratgan ID ni ololmaydi. Sizdagi yechim: parent so‘rov tugashini kutadi, so‘ng ID ni last_item_id propida qaytaradi.

Komponent hodisalari DOM hodisalari kabi barcha ajdod komponentlarga avtomatik ko‘tarilmaydi. Yuqoriroqqa uzatish kerak bo‘lsa, tegishli komponent uni qayta chiqarishi mumkin.

Manba: Vue: Component Events

v-model va defineModel

Bir qiymatni parent ham, child ham o‘zgartirishi kerak bo‘lsa, v-model qulay. Masalan, parent formani ochadi, childdagi yopish tugmasi esa uni yopadi.

Parent tomoni

```
const isOpen = ref(false)

// Template:
<MyForm v-model:open="isOpen" />
```

Child tomoni

```
const open = defineModel<boolean>('open', {
    required: true
})

function closeForm() {
    open.value = false
}

// Template:
<button @click="closeForm">Yopish</button>
```

Parent isOpen ni o‘zgartirsa, child yangi qiymatni oladi. Child open.value ga yozsa, model yangilash hodisasi chiqadi va parentdagi isOpen yangilanadi.

Qisqa yozuvning ochilgan ko‘rinishi

```
<MyForm
    :open="isOpen"
    @update:open="isOpen = $event"
/>
```

Demak, ikki tomonlama aloqa ikkita yo‘ldan iborat: qiymat prop orqali pastga, yangi qiymat esa update:open hodisasi orqali yuqoriga boradi. defineModel shu prop va hodisa uchun qulay ref beradi.

Childdagi defineModel ning o‘zi parent o‘zgaruvchisini ulab qo‘ymaydi. Parent ham v-model ishlatishi yoki update hodisasini qo‘lda tinglashi kerak.

Nomli va nomsiz model

```
defineModel<string>()         // modelValue
defineModel<boolean>('open')  // open

<MyInput v-model="name" />
<MyForm v-model:open="isOpen" />
```

defineModel Vue 3.4+ da mavjud. Misollardagi required: true parent model qiymatini berishini talab qiladi. Default berilib, parent qiymati undefined qolsa, boshlang‘ich qiymatlar farq qilishi mumkin.

Manba: Vue: Component v-model

watch: o‘zgarishga javob berish

Prop ID ni yetkazadi; watch esa shu ID o‘zgarganda qo‘shimcha harakat bajaradi. Sizda bu harakat - tegishli formani ochish.

```
watch(
    () => props.last_item_id,
    (id) => {
        if (!id) return
        open.value = true
        handleOpenForm(id)
    }
)
```

Nega () => props.last_item_id?

Bu funksiya getter deb ataladi: chaqirilganda joriy prop qiymatini qaytaradi. Vue uning ichida qaysi reaktiv qiymat o‘qilganini kuzatadi. Qiymat o‘zgarsa, callbackni ishga tushiradi.

```
// Noto‘g‘ri: string / null qiymat uzatilmoqda
watch(props.last_item_id, callback)

// To‘g‘ri: qiymatni o‘qiydigan funksiya uzatilmoqda
watch(() => props.last_item_id, callback)
```

Birinchi variant taxminan watch("exp-42", callback) deyishga o‘xshaydi. Bu matn o‘zining qayerdan kelganini bildirmaydi. TypeScript uni kuzatish manbasi sifatida qabul qilmaydi.

ref uchun esa getter shart emas

```
const lastId = ref<string | null>(null)
watch(lastId, (id) => { /* ... */ })

// Prop uchun muqobil:
const propId = toRef(props, "last_item_id")
watch(propId, (id) => { /* ... */ })
```

ref o‘zi reaktiv manba. lastId.value esa uning hozirgi qiymati. Shu sabab refni kuzatishda watch(lastId, ...) ishlatiladi.

> Oddiy watch boshlang‘ich qiymat uchun callbackni darhol ishlatmaydi. Agar mount paytida kelgan ID ni ham ochish kerak bo‘lsa, uchinchi argument sifatida { immediate: true } bering.

Bir xil string ID yana tayinlansa, qiymat o‘zgarmagani uchun callback qayta ishlamaydi. Yangi experience uchun yangi ID keladigan jarayonda bu muammo emas. Bu holatga deep: true kerak emas.

Manba: Vue: Watchers

Sizdagi experience qanday ochiladi?

Quyidagi oqim loyiha kodidagi ExperienceSection, ResumeEditorSection va useExperience asosida tushuntirilgan. Namunalardagi o‘zgarishlar tavsiya sifatida berilgan.

1. Parent yangi elementni yaratadi

```
const last_item_id = ref<string | null>(null)

const handleAddSection = async () => {
    const id = await createExperience(resumeStore.resume.id)
    if (id) last_item_id.value = id
}

// Parent template qismi:
<ResumeEditorSection
    @add="handleAddSection"
    :last_item_id="last_item_id"
/>
```

Child emit("add") chiqaradi. createExperience server javobini olgach, experience ro‘yxatiga element qo‘shadi va ID qaytaradi. So‘rov muvaffaqiyatsiz bo‘lsa null qaytadi; if tekshiruvi sabab yangi ID uzatilmaydi.

2. Child ID ni ochiq formalar to‘plamiga yozadi

```
const openedForms = ref<Set<string>>(new Set())

const handleOpenForm = (id: string) => {
    openedForms.value.clear()
    openedForms.value.add(id)
}
```

Set takrorlanmaydigan ID lar to‘plami. clear() oldingi barcha ID larni olib tashlaydi, add(id) esa yangi ID ni kiritadi. Shunday qilib, bir vaqtning o‘zida bitta ichki forma ochiq turadi.

3. Template Set dan ochiqlikni hisoblaydi

```
<UCollapsible :open="openedForms.has(item.id)" />
```

Agar item.id to‘plamda bo‘lsa, has true qaytaradi. Vue ref ichidagi Set o‘zgarishini kuzatadi va open qiymatini yangilaydi. Bir nechta forma ochiq qolsin desangiz, handleOpenForm ichidagi clear() ni olib tashlash yetarli.

Tashqi section va ichki forma ikki alohida holat: open tashqi sectionni boshqaradi, openedForms esa ichki formalarni. Watch ichidagi open.value = true tashqi section ham ochiq bo‘lishini ta’minlaydi.

Teskari aloqa va amaliy tekshiruv

Foydalanuvchi collapsible ni o‘zi ochsa-chi?

```
<UCollapsible
    :open="openedForms.has(item.id)"
    @update:open="
        $event
            ? handleOpenForm(item.id)
            : openedForms.delete(item.id)
    "
/>
```

Bu yerda :open joriy holatni beradi. @update:open esa komponent yuborgan yangi boolean qiymatni qabul qiladi. true kelsa ID qo‘shiladi; false kelsa ID o‘chiriladi. Keyingi renderda has(item.id) yangilangan qiymatni qaytaradi.

openedForms.has(item.id) funksiya natijasi bo‘lgani uchun unga qiymat tayinlab bo‘lmaydi. Shu sabab v-model:open="openedForms.has(item.id)" o‘rniga yuqoridagi prop va hodisa juftligi mos keladi.

Sizdagi add_pending haqida

```
// Childda mavjud:
const add_pending = defineModel<boolean>("add_pending", {
    default: false
})

// Parent hozir qiymatni shunday uzatadi:
<ResumeEditorSection :add_pending="isCreating" />
```

Bu ulanishda parent qiymatni beradi, ammo update:add_pending hodisasini tinglamaydi. Child add_pending ga yozsa, parentdagi isCreating shu yozuv bilan yangilanmaydi. Child faqat loading/disabled uchun o‘qisa, oddiy prop yetarli. Ikki tomondan boshqarish talab qilinsagina v-model tanlang.

O‘zingizni tekshiring

1. Childga faqat sarlavha kerak. Nima ishlatasiz?
2. Child o‘chirish tugmasi bosilganini bildirishi kerak. Nima ishlatasiz?
3. Parent ham, child ham open ni yangilaydi. Nima ishlatasiz?
4. Yangi ID kelganda Set ga qo‘shish kerak. Nima ishlatasiz?

Javoblar: 1 - props. 2 - emit va parentdagi @delete. 3 - v-model va defineModel. 4 - getter bilan watch.

Qiymat qayerda saqlanadi? Uni kim o‘zgartiradi? O‘zgarganda kimga xabar kerak? Shu uch savol qaysi bog‘lanishni tanlashni aniqlaydi.

Asos: suhbatdagi savollar va loyiha fayllari (2026-10-13). Rasmiy Vue havolalari tegishli mavzu sahifalarida berilgan. Qo‘llanma loyiha kodini o‘zgartirmaydi.